

智能家居控制系统

产品总体设计文档

产品名称：OmniHome 智能家居控制系统

文档类型：产品总体设计文档

文档版本：V1.0

编制日期：2026 年 6 月 18 日

目录

- 1 引言
- 2 系统总体设计
- 3 系统架构设计
- 4 功能模块设计
- 5 数据库设计
- 6 接口设计
- 7 运行设计
- 8 安全设计
- 9 部署设计
- 10 总结与展望

1 引言

1.1 编写目的

本文档用于对 OmniHome 智能家居控制系统的总体设计方案进行系统化说明，重点描述系统的分层架构、功能模块、数据组织、接口约束、运行链路、安全机制与部署方式，为课程设计、项目答辩和后续扩展提供统一的技术依据。

1.2 项目背景

随着家庭场景中智能门锁、灯光、空调、摄像头和环境传感器等设备数量持续增加，用户对统一管理、状态感知与联动控制的需求日益突出。传统分散式控制方式存在入口分裂、状态不一致和操作反馈滞后的问题，因此有必要构建一个集中式智能家居控制系统原型。

1.3 设计目标

本系统以 OpenHarmony 客户端为交互入口，以 Fastify 控制中心服务为中枢，以共享数据契约和 SQLite 持久化机制为支撑，目标是在演示环境中实现设备集中控制、场景联动、命令可追踪、状态可同步和系统可扩展的完整闭环。

2 系统总体设计

2.1 系统定位

OmniHome 智能家居控制系统定位为面向家庭用户和家庭成员的集中控制平台。系统并不局限于静态页面展示，而是围绕“设备状态读取、控制命令下发、场景与自动化执行、实时状态回写”构建完整的数据链路，从而支撑原型演示与工程扩展双重目标。

2.2 总体设计原则

系统总体设计遵循分层解耦原则、统一契约原则、缓存与同步并重原则以及安全控制前置原则。前端界面、领域逻辑、接口服务和数据持久层之间通过明确边界进

行协作，既降低了模块间耦合度，也使后续替换模拟设备或扩展真实 IoT 接入时具备较好的延展性。

图2-1 系统总体架构图

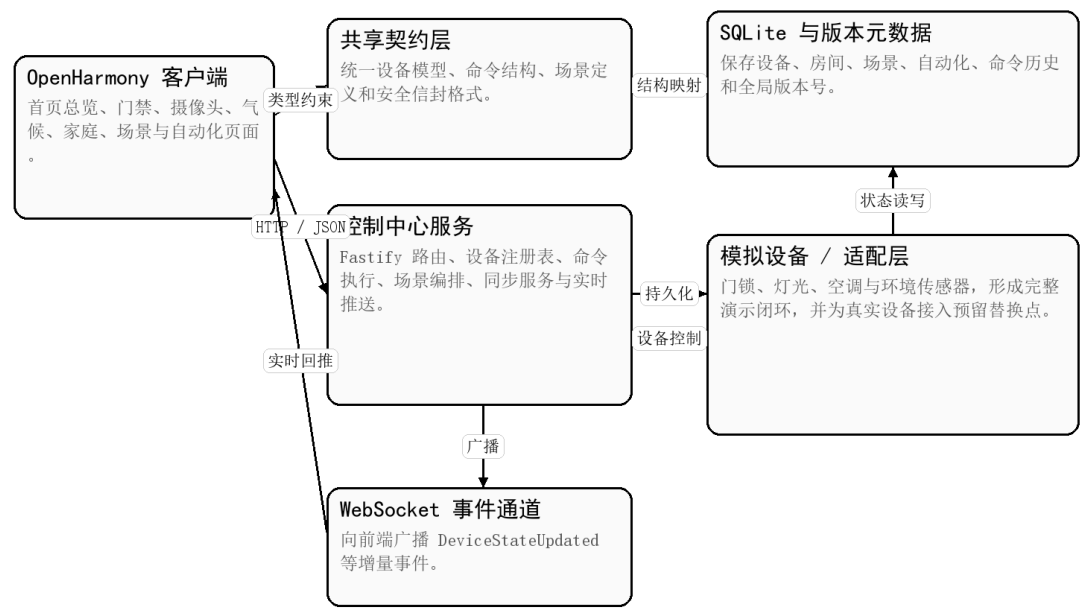


图 2-1 系统总体架构图

2.3 系统边界

系统边界主要包括 OpenHarmony 客户端、控制中心服务、共享设备契约、SQLite 数据库和模拟设备层。其中，客户端负责用户交互与状态呈现；控制中心负责设备管理、命令执行与同步推送；共享契约负责统一前后端语义；数据库负责本地持久化与版本控制；模拟设备层则为原型验证提供可运行支撑。

3 系统架构设计

3.1 总体架构说明

系统采用“客户端应用层 + 控制中心服务层 + 共享契约层 + 数据与设备层”的四层协同结构。OpenHarmony 客户端通过 HTTP 和 WebSocket 与控制中心服务通信，

控制中心服务在内部通过路由模块、注册表、场景管理器、命令历史与数据库服务完成业务编排，数据模型通过 `packages/device-contract` 统一定义。

3.2 前端架构设计

前端以 ArkTS 页面为用户入口，围绕首页总览、门禁、摄像头、气候、家庭、自动化和通知等页面组织功能。其核心设计思想是通过控制层和数据访问层隔离页面渲染逻辑与远程数据逻辑，使界面组件聚焦展示与交互，领域逻辑聚焦状态装配与命令组织，避免出现 UI 代码与网络逻辑强耦合的问题。

3.3 后端架构设计

后端基于 Fastify 构建控制中心服务，在 `buildApp()` 中统一装配 `access`、`camera`、`climate`、`commands`、`devices`、`family`、`rooms`、`scenes`、`automations`、`sync` 和 `websocket` 等业务路由。后端同时维护设备注册表、场景注册表、房间注册表和命令历史，并借助安全信封机制、数据库事务和 WebSocket 广播机制形成控制闭环。

3.4 共享契约设计

共享契约层位于 `packages/device-contract`，集中定义设备描述、设备状态、设备命令、场景定义、命令历史和安全信封等关键模型。通过共享契约，前后端可以围绕统一的数据结构进行开发，降低接口演化造成的联调成本，并为系统未来扩展多终端接入提供语义基础。

4 功能模块设计

系统功能模块围绕家庭集中控制场景展开，既覆盖智能家居核心控制能力，也兼顾状态感知、家庭协同和系统支撑能力。

图4-1 功能模块结构图

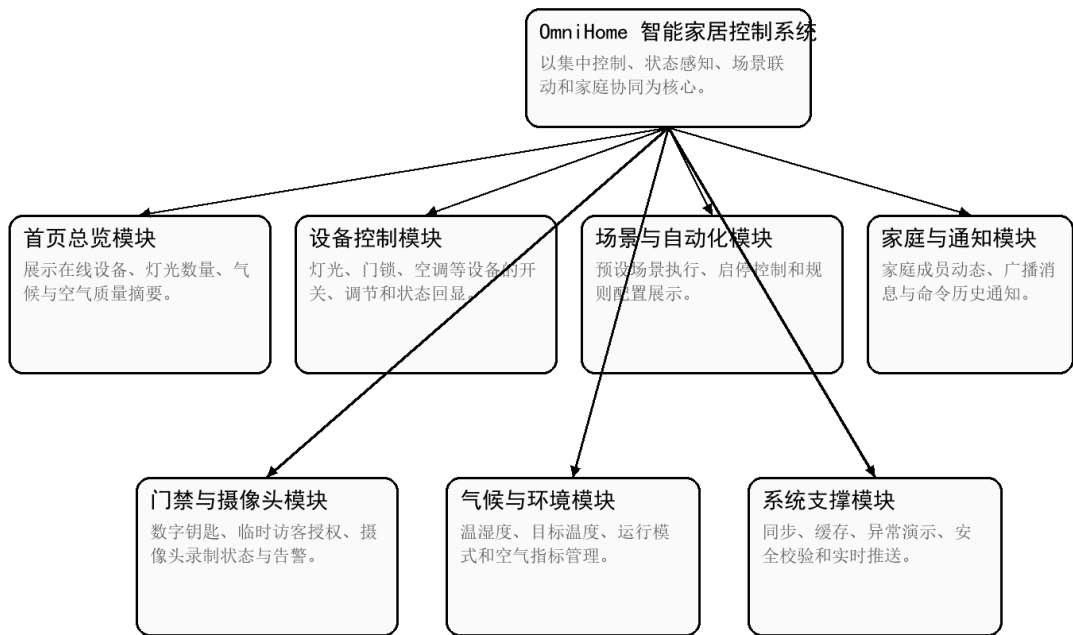


图 4-1 功能模块结构图

4.1 首页总览模块

首页总览模块负责展示全屋设备在线情况、灯光数量、温湿度与空气质量等摘要信息，并提供门禁、摄像头、气候控制等垂直模块的快捷入口，是用户进入系统后的主要信息汇聚页面。

4.2 设备控制模块

设备控制模块面向门锁、灯光、空调等设备提供统一的控制入口，支持开关、温度设定、亮度调节、色温切换和门锁控制等能力。所有控制行为均通过命令接口提交，并在服务端执行后反馈到本地状态。

4.3 场景与自动化模块

场景与自动化模块支持预定义场景的查看、启停和执行，以及自动化规则的创建、启用和维护。该模块的价值在于将多个设备命令组合成可复用的家庭控制策略，提高系统的业务表达能力。

4.4 家庭与通知模块

家庭与通知模块承担家庭成员活动展示、全屋广播以及命令历史回显等功能，使系统不再局限于单一设备管理，而是具备家庭协同与状态留痕能力。

5 数据库设计

5.1 设计目标

数据库设计既服务于服务端状态持久化，也服务于增量同步与命令历史追踪。当前系统使用 SQLite 作为轻量级本地数据库，兼顾开发部署简便性和原型演示稳定性。

5.2 主要数据表

表名	主要字段	设计作用
metadata	key、value	保存 global_version、schema_version 等元数据，用于版本控制与迁移管理。
devices	id、name、type、room_id、state_json、updated_at、version、is_deleted	保存设备状态快照，是首页总览、设备详情和同步接口的重要数据来源。
rooms	id、name、icon、built_in、updated_at、version、is_deleted	保存房间信息，为设备归属和页面组织提供支撑。
scenes	id、name、description、enabled、trigger_json、repeat_json、commands_json	保存场景定义、启用状态及动作集合。
automations	id、icon、name、trigger_type、trigger_json、action_json、enabled、version	保存自动化规则及触发动作配置。
history	id、request_id、device_id、command_name、status、message、created_at	保存命令执行结果和错误反馈，支撑通知与审计。

5.3 数据设计特点

数据库设计中最关键的点在于引入 `global_version` 和各业务表 `version` 字段，用于支撑 `/api/sync` 增量同步接口。每当设备、房间、场景或自动化对象发生变化时，系统通过事务同步提升版本号，从而使前端能够按版本拉取变化数据，避免全量同步带来的性能开销。

6 接口设计

6.1 接口分层说明

系统接口设计围绕业务域进行分组，包括设备与摘要接口、命令控制接口、场景与自动化接口、垂直领域接口、同步接口和实时推送接口。接口分组有助于提高系统可维护性，也便于前端页面按模块装配数据。

接口类别	代表接口	作用说明
设备与摘要接口	GET <code>/api/devices</code> ; GET <code>/api/devices/:deviceId</code> ; GET <code>/api/summary</code>	为首页总览和设备详情提供基础状态数据。
命令控制接口	POST <code>/api/demo/sign-command</code> ; POST <code>/api/commands</code> ; GET <code>/api/commands/history</code>	完成命令签名、命令执行与历史回显。
场景与自动化接口	GET/POST/PUT/DELETE <code>/api/scenes</code> ; GET/POST/PUT/DELETE <code>/api/automations</code>	支撑场景和自动化规则的查询、维护与执行。
垂直领域接口	GET <code>/api/access</code> ; GET <code>/api/cameras</code> ; GET <code>/api/family</code> ; GET <code>/api/climate</code>	为门禁、摄像头、家庭和气候模块提供专属数据。
同步接口	GET <code>/api/sync?lastVersion=n</code>	返回版本号之后的增量数据，用于前端缓存刷新。
实时接口	GET <code>/ws/events</code>	建立 WebSocket 通道，将设备状态变化广播给客户端。

6.2 接口设计特点

命令类接口采用“先签名、后执行”的双阶段设计，避免客户端直接拼装不受保护的命令。同步类接口采用版本号增量拉取策略，与 WebSocket 实时事件共同组成“主动推送 + 按需补偿”的状态一致性机制。

7 运行设计

系统运行设计重点描述从用户操作触发到界面状态回写的全过程。与单纯页面型系统不同，本系统的数据流既包含读取路径，也包含控制路径、同步路径和实时路径。

图7-1 核心控制与同步数据流图

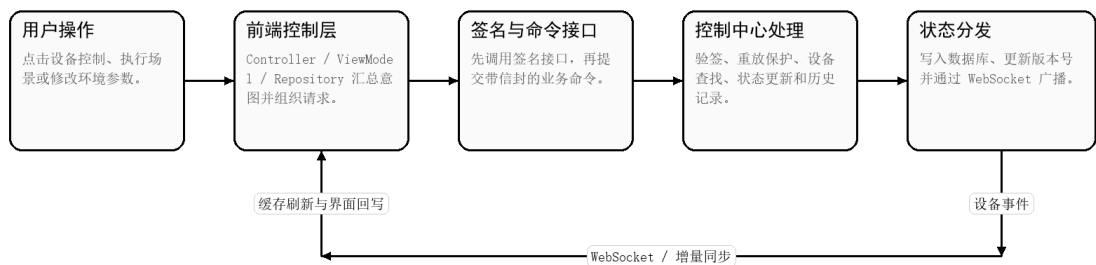


图 7-1 核心控制与同步数据流图

7.1 读取链路

读取链路主要用于页面初始化和状态展示。客户端进入页面后，通过 Repository 发起摘要或领域接口请求，控制中心服务装配数据库状态和注册表状态后返回前端，前端将返回结果映射为页面视图模型并完成渲染。

7.2 控制链路

控制链路用于设备控制和场景执行。用户在前端发起控制行为后，系统首先调用签名接口生成 HMAC 安全信封，再调用命令执行接口。控制中心服务完成验签、重放保护、设备查找和模拟器执行后，更新数据库、写入历史记录，并返回执行结果。

7.3 同步链路

同步链路主要通过 `/api/sync` 实现。前端记录本地 `lastVersion`，后台根据传入版本号查询 `devices`、`rooms`、`scenes` 和 `automations` 中版本更高的数据，将其打包返回，并附带 `currentVersion` 供前端更新本地同步水位。

7.4 实时链路

实时链路通过 `/ws/events` 建立 WebSocket 通道。服务端在命令执行成功、场景执行完成或异常演示状态变化后调用 `broadcastEvent()`，将 `DeviceStateUpdated` 等事件广播给在线客户端，客户端接收后更新缓存和界面，从而减少轮询成本并提升反馈实时性。

8 安全设计

8.1 命令签名机制

系统在命令执行前通过 `/api/demo/sign-command` 对设备命令进行 HMAC-SHA256 签名，生成包含命令、时间戳、随机 `nonce` 和摘要信息的安全信封。客户端随后将完整信封提交到 `/api/commands`，由服务端进行统一验证。

8.2 重放保护机制

服务端通过 `ReplayGuard` 记录 `nonce` 和时间窗口，对重复提交或超时命令进行拒绝，从而降低控制链路受到重放攻击的风险。该机制对于智能家居控制场景尤其重要，因为门锁、灯光和空调控制均属于可执行的物理世界指令。

8.3 异常授权与安全演示

系统提供 `POST /api/demo/faults/security` 接口，可强制切换到命令拒绝演示模式，用于展示命令授权失败时的系统表现。这一设计既体现了原型系统对安全异常的可视化支撑，也有助于答辩时演示安全反馈链路。

8.4 传输安全预留

控制中心服务预留 TLS_CERT_PATH 和 TLS_KEY_PATH 配置项，可在后续部署环境中切换至 HTTPS 监听模式。当前版本重点完成了命令链路和事件链路的安全边界设计，为未来面向真实网络环境部署奠定基础。

9 部署设计

系统部署主要面向开发与演示环境展开，采用“OpenHarmony 客户端 + 本地控制中心服务 + SQLite 数据库 + 模拟设备层”的轻量级部署结构。该结构部署成本低、链路清晰，适合课程答辩和功能验证。

图9-1 系统部署图

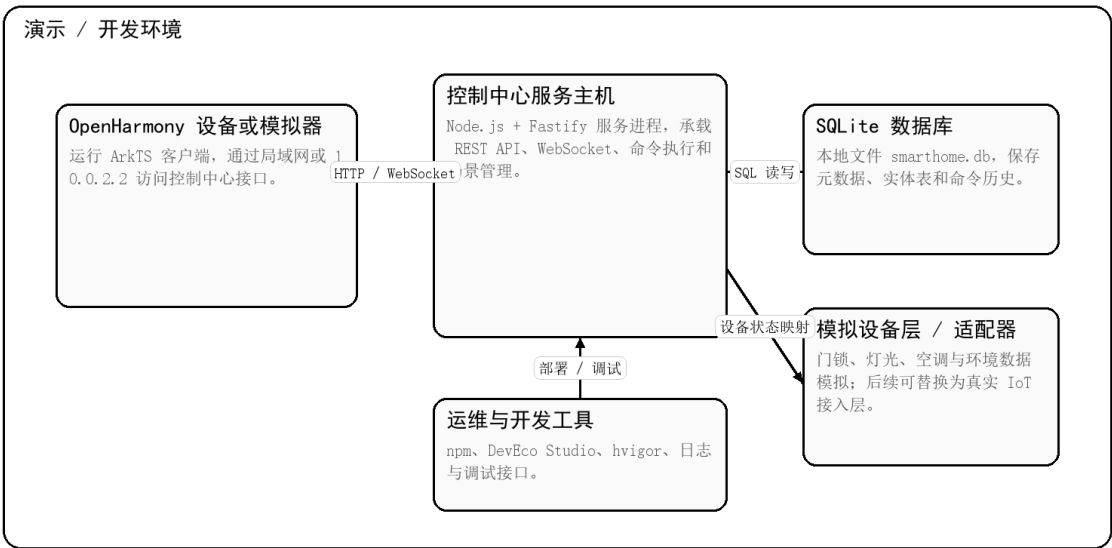


图 9-1 系统部署图

9.1 客户端部署

客户端运行于 OpenHarmony 模拟器或真实设备，通过 DevEco Studio 构建 HAP 包后安装。演示环境下，客户端默认通过 10.0.2.2 访问宿主机控制中心服务；若部署到真机环境，则需要将接口地址调整为局域网主机地址。

9.2 服务端部署

控制中心服务运行于 Node.js 环境，依赖 Fastify、better-sqlite3 和共享契约包。服务端启动后同时提供 REST API 与 WebSocket 通道，数据库文件 smarthome.db 与服务进程共机部署。

9.3 扩展部署方向

后续如果接入真实物联网设备，可在保持上层控制接口不变的前提下，将模拟设备层替换为真实设备网关或驱动适配层；同时可将 SQLite 迁移至独立数据库服务，以增强并发处理能力和部署弹性。

10 总结与展望

本系统在总体设计上形成了较完整的智能家居控制原型架构，既具备集中控制、场景联动、状态同步、历史留痕和安全控制等核心能力，也通过共享契约、版本同步和实时推送机制体现了较清晰的工程化思路。

未来可进一步扩展真实设备接入、用户身份认证、多家庭空间隔离、规则引擎增强和云边协同能力，使系统从课程原型逐步演进为更加贴近真实应用场景的智能家居控制平台。